

به نام خدا



سامانه‌های یادگیری ماشین توزیع شده (پاییز ۱۴۰۴)

تمرین کامپیوتری ۴

مهلت ارسال: ۲۶ دی ماه ۱۴۰۴

---

استاد درس: دکتر دوستی

دستیار طراح: علی رمضان‌ی

---

### قوانین و ملاحظات

#### نحوه ارسال تمرین:

- تمامی فایل‌ها باید در یک فایل فشرده با نام DMLS-CA4-StudentID ارسال شوند.
- کدهای مربوط به هر بخش را بر اساس جدول انتهای همین فایل ذخیره کرده و همراه گزارش ارسال کنید.
- تمامی کدهای ارسال شده باید امکان اجرای مجدد داشته باشند.

#### رعایت اصول آکادمیک و صداقت علمی:

- این تمرین باید به صورت **فردی** انجام شود.
- در صورت مشاهده تشابه در پاسخ، تمامی افراد درگیر نمره صفر دریافت خواهند کرد.

#### استفاده از ابزارهای هوش مصنوعی:

- استفاده از ابزارهایی مانند Gemini، ChatGPT و موارد مشابه مجاز است، اما تحت شرایط زیر:
- نحوه استفاده از این ابزارها را در گزارش خود توضیح دهید.
  - تمامی پرامپت‌ها و لینک‌های استفاده شده را در انتهای گزارش قرار دهید.

لطفاً پیش از شروع کار بر روی تمرین، به نکات زیر توجه فرمایید:

- هدف از انجام این تمرین آموزش و ارزیابی شبکه‌های عصبی به صورت توزیع شده و آشنایی با ابزار پروفایلر PyTorch می‌باشد.
- لطفاً قبل از انجام تمرین ویدیوی مربوطه را مشاهده کنید و لینک‌های داده شده را مطالعه بفرمایید.
- سوال اول و دوم باید بر روی کلاستر درس (بردهای رزبری پای) انجام شود. برای سوال سوم باید از محیط کولب (Colab) استفاده نمایید.
- آدرس بردهای رزبری پای به ترتیب زیر می‌باشد:

172.18.32.200

172.18.32.201

172.18.32.202

172.18.32.203

- دقت کنید که منابع کلاستر بین همه‌ی دانشجویان مشترک است. به همین دلیل سعی کنید بیشتر از مقدار مورد نیاز از آن استفاده ننمایید.
- سوالات خود را در گروه تلگرام درس مطرح نمایید یا فقط از طریق آدرس ایمیل زیر بپرسید:

ali.ramezani.96@ut.ac.ir

- **نکته مهم در مورد داده‌ها:** برای همه سوالات این تمرین از فایل دیتاست داده شده استفاده نمایید. دقت کنید که پوشه‌ی داده شده شامل چهار فایل است که شامل ویژگی‌ها و برجسب‌های داده‌های آموزش و تست است.
- داده‌ها را می‌توانید به کمک کتابخانه‌ی numpy بخوانید. فایل‌های داده‌های ویژگی برای کاهش سایز به صورت float16 ذخیره شده‌اند؛ برای اینکه بتوانید از آن‌ها برای آموزش استفاده کنید لازم است تا آن‌ها را به float32 تبدیل کنید.

۴۰ نمره

### سؤال ۱. آموزش توزیع شده با مدیریت حافظه (Gradient Accumulation)

هدف از این تمرین آشنایی با نحوه استفاده از torchrun برای آموزش شبکه عصبی به صورت توزیع شده به کمک slurm و پیاده‌سازی تکنیک Gradient Accumulation جهت مدیریت حافظه و کاهش سربار ارتباطی است.

#### ۱.۱ طراحی مدل و پیاده‌سازی

مدلی طراحی کنید که شروط زیر را داشته باشد:

- مدلی شامل لایه‌های FeedForward، ReLU و BatchNorm طراحی کرده و بسازید که با آن بتوان به دقت بالای ۸۰ درصد روی داده‌های تست رسید.
- چک‌پوینت مدل را در انتهای آموزش تنها به وسیله‌ی یک پراسس (رنک صفر) ذخیره نمایید.
- به سازگاری Backend ارتباطی با سخت‌افزار کلاستر (CPU-only) دقت کنید.

- **مدیریت حافظه:** فرض کنید حافظه دستگاه محدود است و امکان بارگذاری Batch Size بزرگ (مثلاً ۱۲۸) به صورت یکجا وجود ندارد. به همین منظور کد آموزش را طوری بنویسید تا با Batch Size فیزیکی کوچک (مثلاً ۳۲) اجرا شود، اما با استفاده از تکنیک Gradient Accumulation (تجمع گرادیان) در ۴ مرحله، به Effective Batch Size برابر با ۱۲۸ برسید.

- در پیاده‌سازی Gradient Accumulation، مراقب سربار ارتباطی (Communication Overhead) غیرضروری در گام‌های میانی باشید. آیا لازم است در هر pass گرادیان‌ها sync شوند؟ (در گزارش به این سوال پاسخ دهید).  
در نهایت به کمک torchrun و slurm مدل را در حالت‌های زیر آموزش دهید:

۱. یک ماشین و یک هسته

۲. یک ماشین و دو هسته

۳. دو ماشین و یک هسته

۴. دو ماشین و دو هسته

## ۲.۱ گزارش عملکرد

زمان آموزش و دقت مدل را در هر چهار حالت محاسبه کرده و گزارش کنید.

## ۳.۱ مقایسه بدون تجمع گرادیان

یک بار دیگر مدل را در حالت دو ماشین و دو هسته ولی بدون Gradient Accumulation آموزش دهید و نتایج را با حالت قبل مقایسه کنید.

۴۰ نمره

## سؤال ۲. آموزش و ساده‌سازی با Hugging Face Accelerate

هدف: آشنایی با کتابخانه Hugging Face Accelerate جهت حذف پیچیدگی‌های کدنویسی توزیع‌شده، مدیریت هوشمند حافظه و پیاده‌سازی ساده‌تر تکنیک تجمع گرادیان.  
کتابخانه Accelerate فرآیند راه‌اندازی آموزش توزیع‌شده (مدیریت Device و Rank) را پنهان می‌کند، شما در این سوال، کدی که در سوال یک به صورت دستی نوشتید را بازنویسی و بهینه می‌کنید.

## ۱.۲ بازنویسی کد با Accelerate

برای نوشتن کد این بخش موارد زیر را در نظر بگیرید:

- کد بخش (الف) سوال یک را با استفاده از Accelerate بازنویسی کنید.
- **ساده‌سازی Gradient Accumulation:** در سوال یک، برای پیاده‌سازی تجمع گرادیان مجبور بودید از شرط‌های متعدد و no\_sync استفاده کنید. در این بخش، همان منطق سوال یک (رسیدن به بچ‌سایز موثر ۱۲۸) را این بار با استفاده از کانتکست منیجر accelerator.accumulate پیاده‌سازی کنید.

**نکته مهم:** برای استفاده از این کتابخانه، حتماً دستور زیر را در ابتدای فایل bash خود قرار دهید:

```
source home/shared_files/pytorch_venv/bin/activate
```

در نهایت با کمک slurm و دستور accelerate launch، مدل را در همان ۴ حالت ذکر شده در سوال یک (از یک ماشین/یک هسته تا دو ماشین/دو هسته) آموزش دهید.

## ۲.۲ تحلیل و مقایسه

کدهای سوال یک و دو را مقایسه کنید. Accelerate چگونه پیچیدگی همگام‌سازی گرادینت‌ها در DDP را مدیریت می‌کند؟

## ۳.۲ نتایج

زمان آموزش و دقت مدل را در هر چهار حالت گزارش کنید. نتایج را با سوال یک مقایسه کرده و تاثیر استفاده از Accelerate بر خوانایی کد و سرعت اجرا را تحلیل کنید.

۲۰ نمره

## سؤال ۳. پروفایلینگ (Profiling)

هدف از این تمرین آشنایی با ابزار پروفایلر PyTorch می‌باشد. ابزار پروفایلر مدت زمان و حافظه‌ی مصرفی برای هر ماژول مدل را محاسبه می‌کند.

## ۱.۳ پروفایلینگ لایه‌ها

- داده‌های تست سوال اول را مطابق با توضیحات قبلی لود نمایید و مدلی دقیقاً مشابه به مدل بخش الف قسمت یک لود نمایید.
- مدل را بر روی CPU لود کنید. در این بخش نیازی به آموزش مدل ندارید.
  - مدل را بر روی صد batch از داده‌ی تست پروفایل نمایید.
  - زمان و حافظه‌ی مصرفی هر یک از ماژول‌های Linear، BatchNorm و ReLU را گزارش کرده و با یکدیگر مقایسه نمایید.

## ۲.۳ مقایسه توابع فعال‌ساز

به جای تابع فعال‌ساز ReLU، از توابع فعال‌ساز Sigmoid، Tanh و GeLU استفاده نمایید و زمان و حافظه‌ی مصرفی هر چهار تابع فعال‌ساز را با همدیگر مقایسه نمایید.

جهت تسلط بیشتر بر مباحث این تمرین و آشنایی با ابزارهای استفاده شده، مطالعه لینک‌های زیر پیشنهاد می‌شود:

1. Hugging Face Accelerate Documentation
2. Accelerate CLI Reference
3. Accelerate GitHub Repository
4. Transformers with Accelerate
5. Multi-GPU on Raw PyTorch with Accelerate (DigitalOcean)
6. PyTorch Profiler Recipe

#### نحوه تحویل تمرین کامپیوتری ۴

۱. گزارش پروژه را در قالب فایل PDF تهیه کنید.

۲. تمامی کدهای پایتون (.py) و اسکریپت‌های اجرایی (.sh) را طبق جدول زیر نام‌گذاری کنید.

۳. در نهایت، فایل گزارش و تمامی کدهای نام‌گذاری شده را در یک فایل zip با نام **شماره دانشجویی خود** در سامانه بارگذاری نمایید.

| نام فایل                 | بخش                  | سوال |
|--------------------------|----------------------|------|
| Q1_SMSC.py<br>Q1_SMSC.sh | (یک ماشین و یک هسته) | ۱    |
| Q1_SMMC.py<br>Q1_SMMC.sh | (یک ماشین و دو هسته) |      |
| Q1_MMSC.py<br>Q1_MMSC.sh | (دو ماشین و یک هسته) |      |
| Q1_MMMC.py<br>Q1_MMMC.sh | (دو ماشین و دو هسته) |      |
| Q2_SMSC.py<br>Q2_SMSC.sh | (یک ماشین و یک هسته) | ۲    |
| Q2_SMMC.py<br>Q2_SMMC.sh | (یک ماشین و دو هسته) |      |
| Q2_MMSC.py<br>Q2_MMSC.sh | (دو ماشین و یک هسته) |      |
| Q2_MMMC.py<br>Q2_MMMC.sh | (دو ماشین و دو هسته) |      |
| Profiler.ipynb           | شامل تمامی بخش‌ها    | ۳    |